

Experiment 1: Initialise a new git repository

(Source: Pages 1-3)

```
mkdir program001
cd program001
git init
touch lab.txt
git status
git add .
git commit -m "hello"
```

Experiment 2: Create a new branch and merge into master

(Source: Pages 4-7)

```
git checkout -b cyber
touch new file
git status
git add .
git commit -m "hello"
git checkout master
git merge cyber
```

Experiment 3: Stash changes, switch branches, and apply stash

(Source: Pages 8-11)

```
mkdir pp3
cd pp3
git init
echo "adding file" > cyber.txt
git status
git add .
git commit -m "hello"
echo "adding content to the file" > cyber.txt
git stash
git stash list
echo "creating new branch with file name lab" > lab.txt
git status
git add .
git commit -m "created a new branch"
echo "listing lab program" > lab.txt
git stash
git stash list
```

Experiment 4: Clone a remote git repository

(Source: Pages 12-13)

```
mkdir pp4
cd pp4
```

```
git init
git clone https://github.com/rohinimudhale/pp4.git
```

Experiment 5: Fetch changes and rebase local branch
(Source: Pages 14-17)

```
cd c:
git clone https://github.com/ambikanelwade/pro5.git
cd pro5
git status
git branch
git branch -r
git fetch
git checkout -b branch-name
git checkout main
touch fetch.txt
git add .
git status
touch fetch1.txt
git add .
git status
git commit -m "adding fetch.txt"
git checkout branch-name
git rebase main
```

Experiment 6: Merge Feature-branch into master with custom message
(Source: Pages 18-21)

```
mkdir por6
cd por6
git init
touch demo.txt
git add .
git commit -m "adding a file demo"
git checkout -b new-branch
touch cyber.txt
git status
git add .
git commit -m "adding a file cyber"
git checkout master
git merge new-branch -m "hello"
```

Experiment 7: Create a lightweight tag and push to remote
(Source: Pages 22-24)

Note: This section in the document reuses the "tag-pro1" repository.
mkdir tag-pro1

```
cd tag-pro1
git init
touch test.txt
git status
git add .
git commit -m "hello"
git tag v1.0
git tag -a v2.0 -m "tags"
git show
git remote -v
git remote add origin https://github.com/rohinimudhale/tag-pro1.git
git push -u origin v1.0
```

Experiment 8: Cherry-pick a commit from a source branch

(Source: Pages 25-29)

```
mkdir program08
cd program08
git init
touch file.txt
git add .
git commit -m "hello"
git checkout -b sub-branch
touch file1.txt
git add .
git commit -m "adding file"
touch file2.txt
git add .
git commit -m "adding the txt file"
git log
git checkout master
git cherry-pick caed7a943ff048de89bd7da56e04b1db008b6610
```

Experiment 9: Use Commit ID to view details

(Source: Pages 30-33)

```
mkdir git-status
cd git-status
git init
touch show.txt
git status
git add .
git commit -m "showing commit id"
touch showsecond.txt
git status
git add .
```

```
git commit -m "showing second commit id"
git log
git show fca984ba3f86a8c6516dd96dad3f697cbe4f3211
```

Experiment 10: List commits by specific author

(Source: Pages 34-36)

```
git clone https://github.com/rohinimudhale/program-10.git
cd program-10
git status
git config user.name "john doe"
touch file.txt
git status
git add .
git commit -m "replacing the user name"
git checkout -b new-branch
git log
```

Experiment 11: Display the last five commits

(Source: Pages 37-41)

```
mkdir git-log1
cd git-log1
git init
touch file.txt
git status
git add .
git commit -m "adding one commit"
touch file2.txt
git status
git add .
git commit -m "adding second commit"
touch file3.txt
git add .
git commit -m "adding third commit"
touch file4.txt
git status
git add .
git commit -m "adding fourth commit"
touch file5.txt
git status
git add .
git commit -m "adding fifth commit"
touch file6.txt
git status
git add .
```

```
git commit -m "adding six commit"  
git log -5
```

Experiment 12: Undo changes using git revert
(Source: Pages 42-43)

```
mkdir git--revert2  
cd git--revert2  
git init  
touch file.txt  
git add .  
git commit -m "reverting the commits"  
git log -n1  
git revert 6f3f91ded8142f2833ae79b3d6636117aab7f5ff --no-edit
```